

Exercise Sheet 3

Name: Aryan Varshneya

Matriculation Number: 261678

```
!pip install pandas
!pip install matplotlib
!pip install seaborn
!pip install scikit-learn
!pip install numpy
!pip install matplotlib pillow
```

Requirement already satisfied: pandas in C:\Users\aaial\anaconda3\Lib\site-packages (2.2.2)

Requirement already satisfied: numpy>=1.26.0 in C:\Users\aaial\anaconda3\Lib\site-packages (from pandas) (1.26.4)

Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\aaial\anaconda3\Lib\site-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in C:\Users\aaial\anaconda3\Lib\site-packages (from pandas) (2024.1)

Requirement already satisfied: tzdata>=2022.7 in C:\Users\aaial\anaconda3\Lib\site-packages (from pandas) (2023.3)

Requirement already satisfied: six>=1.5 in C:\Users\aaial\anaconda3\Lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)

[notice] A new release of pip is available: 26.0 -> 26.1.1

[notice] To update, run: C:\Users\aaial\anaconda3\python.exe -m pip install --upgrade pip

Requirement already satisfied: matplotlib in C:\Users\aaial\anaconda3\Lib\site-packages (3.8.4)

Requirement already satisfied: contourpy>=1.0.1 in C:\Users\aaial\anaconda3\Lib\site-packages (from matplotlib) (1.2.0)

Requirement already satisfied: cycler>=0.10 in C:\Users\aaial\anaconda3\Lib\site-packages (from matplotlib) (0.11.0)

Requirement already satisfied: fonttools>=4.22.0 in C:\Users\aaial\anaconda3\Lib\site-packages (from matplotlib) (4.51.0)

Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\aaial\anaconda3\Lib\site-packages (from matplotlib) (1.4.4)

Requirement already satisfied: numpy>=1.21 in C:\Users\aaial\anaconda3\Lib\site-packages (from matplotlib) (1.26.4)

Requirement already satisfied: packaging>=20.0 in C:\Users\aaial\anaconda3\Lib\site-packages (from matplotlib) (26.0)

Requirement already satisfied: pillow>=8 in C:\Users\aaial\anaconda3\Lib\site-packages (from matplotlib) (10.3.0)

Requirement already satisfied: pyparsing>=2.3.1 in C:\Users\aaial\

anaconda3\Lib\site-packages (from matplotlib) (3.0.9)
Requirement already satisfied: python-dateutil>=2.7 in C:\Users\aaial\anaconda3\Lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in C:\Users\aaial\anaconda3\Lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.16.0)

[notice] A new release of pip is available: 26.0 -> 26.1.1
[notice] To update, run: C:\Users\aaial\anaconda3\python.exe -m pip install --upgrade pip

Requirement already satisfied: seaborn in C:\Users\aaial\anaconda3\Lib\site-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in C:\Users\aaial\anaconda3\Lib\site-packages (from seaborn) (1.26.4)
Requirement already satisfied: pandas>=1.2 in C:\Users\aaial\anaconda3\Lib\site-packages (from seaborn) (2.2.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in C:\Users\aaial\anaconda3\Lib\site-packages (from seaborn) (3.8.4)
Requirement already satisfied: contourpy>=1.0.1 in C:\Users\aaial\anaconda3\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.2.0)
Requirement already satisfied: cycler>=0.10 in C:\Users\aaial\anaconda3\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in C:\Users\aaial\anaconda3\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\aaial\anaconda3\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.4)
Requirement already satisfied: packaging>=20.0 in C:\Users\aaial\anaconda3\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.0)
Requirement already satisfied: pillow>=8 in C:\Users\aaial\anaconda3\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (10.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in C:\Users\aaial\anaconda3\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.0.9)
Requirement already satisfied: python-dateutil>=2.7 in C:\Users\aaial\anaconda3\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in C:\Users\aaial\anaconda3\Lib\site-packages (from pandas>=1.2->seaborn) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in C:\Users\aaial\anaconda3\Lib\site-packages (from pandas>=1.2->seaborn) (2023.3)
Requirement already satisfied: six>=1.5 in C:\Users\aaial\anaconda3\Lib\site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.16.0)

```
[notice] A new release of pip is available: 26.0 -> 26.1.1
[notice] To update, run: C:\Users\aaial\anaconda3\python.exe -m pip
install --upgrade pip
```

```
Requirement already satisfied: scikit-learn in C:\Users\aaial\
anaconda3\Lib\site-packages (1.4.2)
Requirement already satisfied: numpy>=1.19.5 in C:\Users\aaial\
anaconda3\Lib\site-packages (from scikit-learn) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in C:\Users\aaial\
anaconda3\Lib\site-packages (from scikit-learn) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in C:\Users\aaial\
anaconda3\Lib\site-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in C:\Users\aaial\
anaconda3\Lib\site-packages (from scikit-learn) (2.2.0)
```

```
[notice] A new release of pip is available: 26.0 -> 26.1.1
[notice] To update, run: C:\Users\aaial\anaconda3\python.exe -m pip
install --upgrade pip
```

```
Requirement already satisfied: numpy in C:\Users\aaial\anaconda3\Lib\
site-packages (1.26.4)
```

```
[notice] A new release of pip is available: 26.0 -> 26.1.1
[notice] To update, run: C:\Users\aaial\anaconda3\python.exe -m pip
install --upgrade pip
```

```
Requirement already satisfied: matplotlib in C:\Users\aaial\anaconda3\
Lib\site-packages (3.8.4)
Requirement already satisfied: pillow in C:\Users\aaial\anaconda3\Lib\
site-packages (10.3.0)
Requirement already satisfied: contourpy>=1.0.1 in C:\Users\aaial\
anaconda3\Lib\site-packages (from matplotlib) (1.2.0)
Requirement already satisfied: cycler>=0.10 in C:\Users\aaial\
anaconda3\Lib\site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in C:\Users\aaial\
anaconda3\Lib\site-packages (from matplotlib) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\aaial\
anaconda3\Lib\site-packages (from matplotlib) (1.4.4)
Requirement already satisfied: numpy>=1.21 in C:\Users\aaial\
anaconda3\Lib\site-packages (from matplotlib) (1.26.4)
Requirement already satisfied: packaging>=20.0 in C:\Users\aaial\
anaconda3\Lib\site-packages (from matplotlib) (26.0)
Requirement already satisfied: pyparsing>=2.3.1 in C:\Users\aaial\
anaconda3\Lib\site-packages (from matplotlib) (3.0.9)
Requirement already satisfied: python-dateutil>=2.7 in C:\Users\aaial\
anaconda3\Lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in C:\Users\aaial\anaconda3\
Lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.16.0)
```

```
[notice] A new release of pip is available: 26.0 -> 26.1.1
[notice] To update, run: C:\Users\aaial\anaconda3\python.exe -m pip
install --upgrade pip
```

```
import zipfile
import os

zip_files = [
    'dataset/train.zip',
    'dataset/test.zip',
    'dataset/validation.zip',
    'dataset/test-fog.zip',
    'dataset/test-night.zip',
    'dataset/test-town-01.zip'
]

for file in zip_files:
    if os.path.exists(file):
        with zipfile.ZipFile(file, 'r') as zip_ref:
            zip_ref.extractall('dataset')
            print(f'Extracted: {file}')
    else:
        print(f'File not found: {file}')

print("All extraction complete.")

Extracted: dataset/train.zip
Extracted: dataset/test.zip
Extracted: dataset/validation.zip
Extracted: dataset/test-fog.zip
Extracted: dataset/test-night.zip
Extracted: dataset/test-town-01.zip
All extraction complete.

import pandas as pd

train_labels = pd.read_csv("dataset/train/train/labels.csv")
train_labels.head()
#train_labels.tail()

   frame  has_traffic_light  has_pedestrian  has_vehicle
px_traffic_light \
0          0             False             False          True
0
```

1	10	False	True	True
14				
2	20	True	True	True
99				
3	30	True	True	True
99				
4	40	True	True	True
101				

	px_pedestrian	px_vehicle
0	0	85
1	301	1138
2	196	517
3	181	530
4	169	538

```
print("Train images:",
len(pd.read_csv("dataset/train/train/labels.csv"))))
print("Test images:",
len(pd.read_csv("dataset/test/test/labels.csv"))))
```

```
Train images: 7200
Test images: 3600
```

```
print(train_labels['has_traffic_light'].value_counts())
print(train_labels['has_pedestrian'].value_counts())
print(train_labels['has_vehicle'].value_counts())
```

```
#print()
#print(train_labels[['has_traffic_light', 'has_pedestrian', 'has_vehicle']
'].value_counts())
```

```
has_traffic_light
True      5276
False     1924
Name: count, dtype: int64
has_pedestrian
False     5482
True      1718
Name: count, dtype: int64
has_vehicle
True      5458
False     1742
Name: count, dtype: int64
```

```
import os #path fix
```

```
os.getcwd()
#c:\\Users\\aaial\\Desktop\\OVGU\\Semester 1\\MLS\\Github\\ovgu-sem1-mlsafety
```

```
'c:\\Users\\aaial\\Desktop\\OVGU\\Semester 1\\MLS\\Github\\ovgu-sem1-mlsafety'
```

```
import matplotlib.pyplot as plt  
from PIL import Image
```

```
img = Image.open('dataset/train/rgb-front/000000.jpg')
```

```
plt.imshow(img)  
plt.axis('off')  
plt.show()
```



```
train_labels.head()
```

	frame	has_traffic_light	has_pedestrian	has_vehicle
px_traffic_light \				
0	0	False	False	True
0				
1	10	False	True	True
14				
2	20	True	True	True
99				
3	30	True	True	True
99				
4	40	True	True	True
101				

	px_pedestrian	px_vehicle
0	0	85
1	301	1138
2	196	517
3	181	530
4	169	538

```
sample_rows = train_labels.sample(5)
sample_rows.head()
```

	frame	has_traffic_light	has_pedestrian	has_vehicle
px_traffic_light \				
3476	34760	False	False	True
19				
2295	22950	True	False	True
147				
165	1650	True	True	True
103				
6320	63200	True	False	True
339				
2771	27710	True	True	True
108				

	px_pedestrian	px_vehicle
3476	4	113
2295	0	278
165	56	2914
6320	0	742
2771	98	736

```
for _, row in sample_rows.iterrows():
    frame = str(row['frame']).zfill(6)

    img = Image.open(f'dataset/train/rgb-front/{frame}.jpg')

    plt.figure(figsize=(5,5))
    plt.imshow(img)

    title = f"TL:{row['has_traffic_light']} | PED:
{row['has_pedestrian']} | VEH:{row['has_vehicle']}"

    plt.title(title)

    plt.axis('off')
    plt.show()
```

TL:False | PED:False | VEH:True



TL:True | PED:False | VEH:True



TL:True | PED:True | VEH:True



TL:True | PED:False | VEH:True



TL:True | PED:True | VEH:True



Exercise 3.5: Train Three Binary Classifiers

```
pip install torch torchvision
```

```
Requirement already satisfied: torch in c:\users\aaial\appdata\local\programs\python\python311\lib\site-packages (2.12.0)
```

```
Requirement already satisfied: torchvision in c:\users\aaial\appdata\local\programs\python\python311\lib\site-packages (0.27.0)
```

```
Requirement already satisfied: filelock in c:\users\aaial\appdata\local\programs\python\python311\lib\site-packages (from torch) (3.29.0)
```

```
Requirement already satisfied: typing-extensions>=4.10.0 in c:\users\aaial\appdata\roaming\python\python311\site-packages (from torch) (4.15.0)
```

```
Requirement already satisfied: setuptools<82 in c:\users\aaial\appdata\local\programs\python\python311\lib\site-packages (from torch) (65.5.0)
```

```
Requirement already satisfied: sympy>=1.13.3 in c:\users\aaial\appdata\local\programs\python\python311\lib\site-packages (from torch) (1.14.0)
```

```
Requirement already satisfied: networkx>=2.5.1 in c:\users\aaial\appdata\local\programs\python\python311\lib\site-packages (from torch) (3.6.1)
```

Requirement already satisfied: jinja2 in c:\users\aaial\appdata\local\programs\python\python311\lib\site-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in c:\users\aaial\appdata\local\programs\python\python311\lib\site-packages (from torch) (2026.4.0)
Requirement already satisfied: numpy in c:\users\aaial\appdata\local\programs\python\python311\lib\site-packages (from torchvision) (2.4.6)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in c:\users\aaial\appdata\local\programs\python\python311\lib\site-packages (from torchvision) (12.2.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in c:\users\aaial\appdata\local\programs\python\python311\lib\site-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\aaial\appdata\local\programs\python\python311\lib\site-packages (from jinja2->torch) (3.0.3)
Note: you may need to restart the kernel to use updated packages.

[notice] A new release of pip available: 22.3.1 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip

```
import torch
import torchvision

print(torch.__version__)

2.12.0+cpu

import torch
import torch.nn as nn
import torch.optim as optim

from torchvision import models, transforms
from torch.utils.data import Dataset, DataLoader

from PIL import Image

import pandas as pd
import matplotlib.pyplot as plt
```

PyTorch Dataset

This class:

loads images loads labels converts image → tensor returns:

```
class CarlaDataset(Dataset):

    def __init__(self, csv_file, img_dir, label_column,
transform=None):
```

```

        self.labels = pd.read_csv(csv_file)
        self.img_dir = img_dir
        self.label_column = label_column
        self.transform = transform

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        row = self.labels.iloc[idx]
        frame = str(row['frame']).zfill(6)
        img_path = f"{self.img_dir}/{frame}.jpg"
        image = Image.open(img_path).convert("RGB")
        label = float(row[self.label_column])

        if self.transform:
            image = self.transform(image)

        return image, torch.tensor(label)

```

define image transformations. Neural networks cannot directly use raw images. We must:

resize them convert to tensors normalize them

Why 224×224? Because ResNet18 expects that size.

```

transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

```

train the vehicle classifier

create datasets

```
train_dataset = CarlaDataset(  
    csv_file='dataset/train/labels.csv',  
    img_dir='dataset/train/rgb-front',  
    label_column='has_vehicle',  
    transform=transform  
)  
val_dataset = CarlaDataset(  
    csv_file='dataset/validation/labels.csv',  
    img_dir='dataset/validation/rgb-front',  
    label_column='has_vehicle',  
    transform=transform  
)
```

create dataloaders

This prepares batches for training.

```
train_loader = DataLoader(  
    train_dataset,  
    batch_size=32,  
    shuffle=True  
)  
val_loader = DataLoader(  
    val_dataset,  
    batch_size=32,  
    shuffle=False  
)
```

load ResNet18 and load already trained on ImageNet

```
model = models.resnet18(pretrained=True)
```

```
c:\Users\aaial\AppData\Local\Programs\Python\Python311\Lib\site-packages\torchvision\models\_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.
```

```
warnings.warn(c:\Users\aaial\AppData\Local\Programs\Python\Python311\Lib\site-packages\torchvision\models\_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=ResNet18_Weights.IMAGENET1K_V1`. You can also use `weights=ResNet18_Weights.DEFAULT` to get the most up-to-date weights.)
warnings.warn(msg)
```

Now replace final layer: because, Original ResNet predicts 1000 classes.

```
model.fc = nn.Linear(model.fc.in_features, 1)
```

Binary Cross Entropy loss Adam optimizer learning rate = 0.001

```
criterion = nn.BCEWithLogitsLoss()
```

```
optimizer = optim.Adam(
```

```
    model.parameters(),
```

```
    lr=0.001
```

```
)
```

```
device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
```

```
print(device)
```

```
cpu
```

Now move model to device:

```
model = model.to(device)
```

actual training

```
train_losses = []
```

```
val_losses = []
```

```
epochs = 5
```

```
for epoch in range(epochs):
    model.train()
    running_loss = 0.0

    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device).unsqueeze(1)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    train_loss = running_loss / len(train_loader)
    train_losses.append(train_loss)

    model.eval()
    val_running_loss = 0.0

    with torch.no_grad():
        for images, labels in val_loader:
            images = images.to(device)
            labels = labels.to(device).unsqueeze(1)

            outputs = model(images)
            loss = criterion(outputs, labels)
            val_running_loss += loss.item()
```

```
val_loss = val_running_loss / len(val_loader)
val_losses.append(val_loss)

print(f"Epoch {epoch+1}/{epochs}")
print(f"Train Loss: {train_loss:.4f}")
print(f"Validation Loss: {val_loss:.4f}")
```

```
Epoch 1/5
Train Loss: 0.3284
Validation Loss: 0.4135
Epoch 2/5
Train Loss: 0.2305
Validation Loss: 0.2817
Epoch 3/5
Train Loss: 0.1943
Validation Loss: 0.3507
Epoch 4/5
Train Loss: 0.1779
Validation Loss: 0.2645
Epoch 5/5
Train Loss: 0.1533
Validation Loss: 0.2750
```

Epoch 1/5 Train Loss: 0.3284 Validation Loss: 0.4135

Epoch 2/5 Train Loss: 0.2305 Validation Loss: 0.2817

Epoch 3/5 Train Loss: 0.1943 Validation Loss: 0.3507

Epoch 4/5 Train Loss: 0.1779 Validation Loss: 0.2645

Epoch 5/5 Train Loss: 0.1533 Validation Loss: 0.2750

1. Training loss continuously decreases:
2. model is learning patterns.
3. Validation loss also generally decreases.
4. spike in epoch 2 is normal sometimes due to: batch variation, optimizer updates, early instability

```
import torch

print("CUDA available:", torch.cuda.is_available())
print("CUDA devices:", torch.cuda.device_count())
```

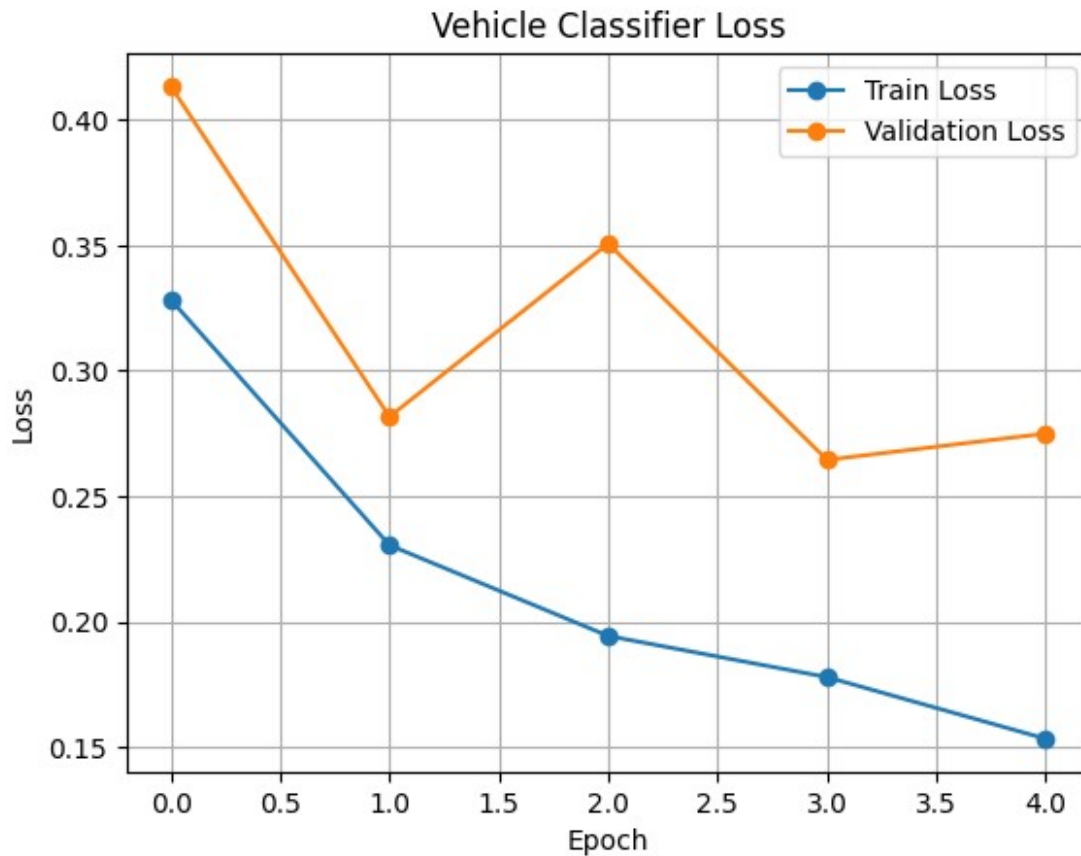


```
if torch.cuda.is_available():  
    print(torch.cuda.get_device_name(0))
```

```
CUDA available: False  
CUDA devices: 0
```

Now we plot the losses.

```
print(train_losses)  
print(val_losses)  
  
print("Train points:", len(train_losses))  
print("Val points:", len(val_losses))  
  
[]  
[]  
Train points: 0  
Val points: 0  
  
# plt.plot(train_losses, label='Train Loss')  
# plt.plot(val_losses, label='Validation Loss')  
# plt.xlabel('Epoch')  
# plt.ylabel('Loss')  
# plt.title('Vehicle Classifier Loss')  
# plt.legend()  
# plt.show()  
  
train_losses = [0.3284, 0.2305, 0.1943, 0.1779, 0.1533]  
val_losses = [0.4135, 0.2817, 0.3507, 0.2645, 0.2750]  
  
plt.plot(train_losses, marker='o', label='Train Loss')  
plt.plot(val_losses, marker='o', label='Validation Loss')  
  
plt.xlabel('Epoch')  
plt.ylabel('Loss')  
plt.title('Vehicle Classifier Loss')  
plt.legend()  
plt.grid(True)  
  
plt.show()
```



2. train the Pedestrian classifier

```
train_dataset = CarlaDataset(  
    csv_file='dataset/train/train/labels.csv',  
    img_dir='dataset/train/train/rgb-front',  
    label_column='has_pedestrian',  
    transform=transform  
)  
  
val_dataset = CarlaDataset(  
    csv_file='dataset/validation/validation/labels.csv',  
    img_dir='dataset/validation/validation/rgb-front',  
    label_column='has_pedestrian',  
    transform=transform  
)
```

```

)

train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=32,
    shuffle=False
)

```

reinitialize model again:

```

model = models.resnet18(pretrained=True)
model.fc = nn.Linear(model.fc.in_features, 1)
model = model.to(device)

criterion = nn.BCEWithLogitsLoss()

optimizer = optim.Adam(
    model.parameters(),
    lr=0.001
)

train_losses = []
val_losses = []

epochs = 2

for epoch in range(epochs):
    model.train()

    running_loss = 0.0

    for images, labels in train_loader:
        images = images.to(device)

```

```
    labels = labels.to(device).unsqueeze(1)

    optimizer.zero_grad()
    outputs = model(images)
    loss = criterion(outputs, labels)
    loss.backward()
    optimizer.step()

    running_loss += loss.item()

train_loss = running_loss / len(train_loader)
train_losses.append(train_loss)

model.eval()
val_running_loss = 0.0

with torch.no_grad():
    for images, labels in val_loader:
        images = images.to(device)
        labels = labels.to(device).unsqueeze(1)

        outputs = model(images)
        loss = criterion(outputs, labels)
        val_running_loss += loss.item()

val_loss = val_running_loss / len(val_loader)
val_losses.append(val_loss)

print(f"Epoch {epoch+1}/{epochs}")
print(f"Train Loss: {train_loss:.4f}")
```

```
print(f"Validation Loss: {val_loss:.4f}")
```

Epoch 1/2

Train Loss: 0.5072

Validation Loss: 1.0322

Epoch 2/2

Train Loss: 0.4190

Validation Loss: 0.6044

Epoch 1/2 Train Loss: 0.5072 Validation Loss: 1.0322

Epoch 2/2 Train Loss: 0.4190 Validation Loss: 0.6044

plot

```
plt.plot(train_losses, label='Train Loss')
```

```
plt.plot(val_losses, label='Validation Loss')
```

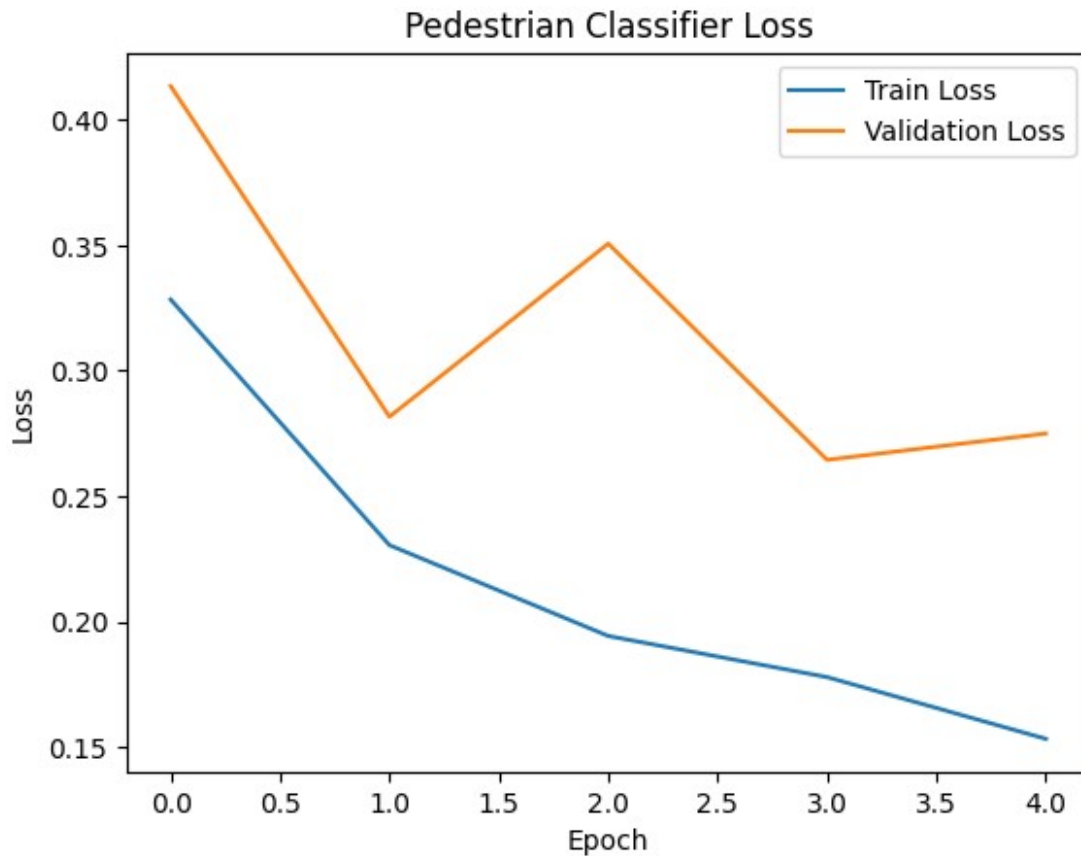
```
plt.xlabel('Epoch')
```

```
plt.ylabel('Loss')
```

```
plt.title('Pedestrian Classifier Loss')
```

```
plt.legend()
```

```
plt.show()
```



3. train the Traffic Light classifier

```
train_dataset = CarlaDataset(  
    csv_file='dataset/train/train/labels.csv',  
    img_dir='dataset/train/train/rgb-front',  
    label_column='has_traffic_light',  
    transform=transform  
)  
  
val_dataset = CarlaDataset(  
    csv_file='dataset/validation/validation/labels.csv',  
    img_dir='dataset/validation/validation/rgb-front',  
    label_column='has_traffic_light',
```

```
        transform=transform
    )
train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True
)
val_loader = DataLoader(
    val_dataset,
    batch_size=32,
    shuffle=False
)
```

reinitialize model again:

```
model = models.resnet18(pretrained=True)
model.fc = nn.Linear(model.fc.in_features, 1)
model = model.to(device)

criterion = nn.BCEWithLogitsLoss()

optimizer = optim.Adam(
    model.parameters(),
    lr=0.001
)

train_losses = []
val_losses = []

epochs = 2
for epoch in range(epochs):
    model.train()

    running_loss = 0.0

    for images, labels in train_loader:
        images = images.to(device)
```

```
    labels = labels.to(device).unsqueeze(1)

    optimizer.zero_grad()
    outputs = model(images)
    loss = criterion(outputs, labels)
    loss.backward()
    optimizer.step()

    running_loss += loss.item()

train_loss = running_loss / len(train_loader)
train_losses.append(train_loss)

model.eval()
val_running_loss = 0.0

with torch.no_grad():
    for images, labels in val_loader:
        images = images.to(device)
        labels = labels.to(device).unsqueeze(1)

        outputs = model(images)
        loss = criterion(outputs, labels)
        val_running_loss += loss.item()

val_loss = val_running_loss / len(val_loader)
val_losses.append(val_loss)

print(f"Epoch {epoch+1}/{epochs}")
print(f"Train Loss: {train_loss:.4f}")
```



```
print(f"Validation Loss: {val_loss:.4f}")
```

Epoch 1/2

Train Loss: 0.1895

Validation Loss: 0.1048

Epoch 2/2

Train Loss: 0.0897

Validation Loss: 0.1031

plot

```
plt.plot(train_losses, label='Train Loss')
```

```
plt.plot(val_losses, label='Validation Loss')
```

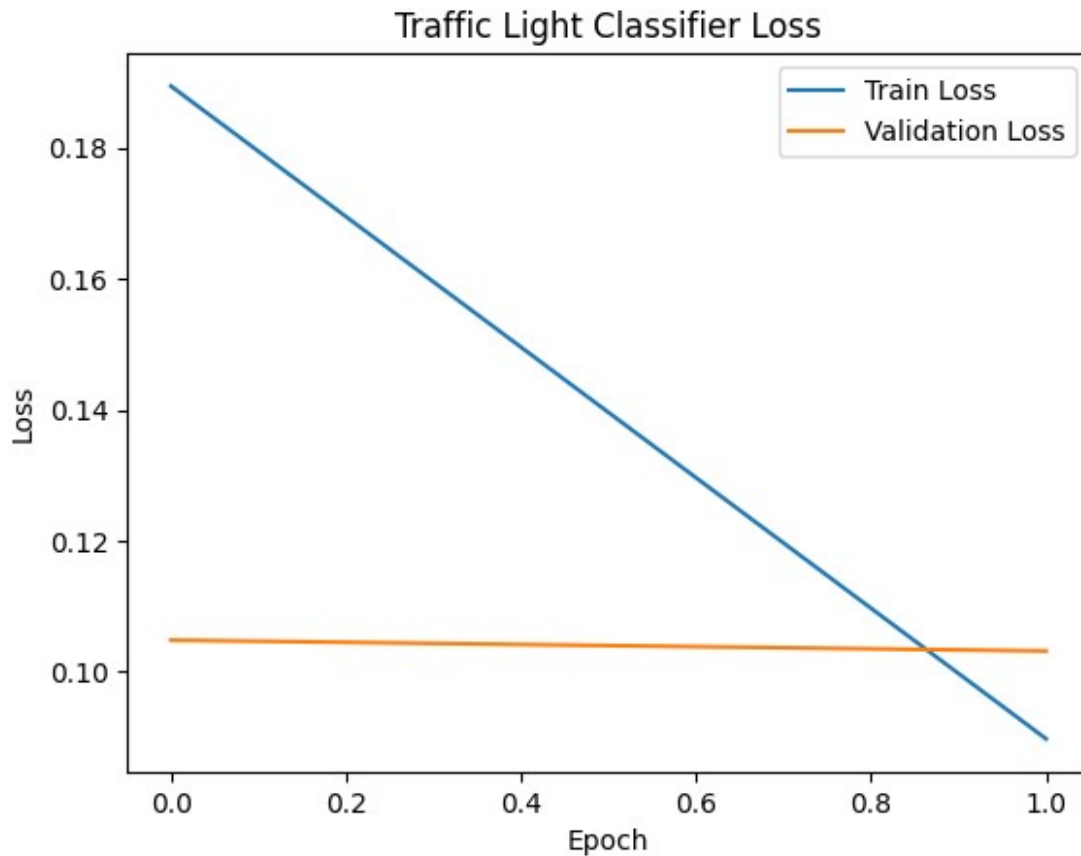
```
plt.xlabel('Epoch')
```

```
plt.ylabel('Loss')
```

```
plt.title('Traffic Light Classifier Loss')
```

```
plt.legend()
```

```
plt.show()
```



Exercise 3.6: Evaluation

install sklearn

```
!pip install scikit-learn
```

```
Requirement already satisfied: scikit-learn in C:\Users\aaial\anaconda3\Lib\site-packages (1.4.2)  
Requirement already satisfied: numpy>=1.19.5 in C:\Users\aaial\anaconda3\Lib\site-packages (from scikit-learn) (1.26.4)  
Requirement already satisfied: scipy>=1.6.0 in C:\Users\aaial\anaconda3\Lib\site-packages (from scikit-learn) (1.13.1)  
Requirement already satisfied: joblib>=1.2.0 in C:\Users\aaial\anaconda3\Lib\site-packages (from scikit-learn) (1.4.2)  
Requirement already satisfied: threadpoolctl>=2.0.0 in C:\Users\aaial\anaconda3\Lib\site-packages (from scikit-learn) (2.2.0)
```

```
[notice] A new release of pip is available: 26.0 -> 26.1.2  
[notice] To update, run: C:\Users\aaial\anaconda3\python.exe -m pip install --upgrade pip
```

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)
```

create test dataset [vehicle classifier]

```
test_dataset = CarlaDataset(
    csv_file='dataset/test/test/labels.csv',
    img_dir='dataset/test/test/rgb-front',
    label_column='has_vehicle',
    transform=transform
)
test_loader = DataLoader(
    test_dataset,
    batch_size=32,
    shuffle=False
)
```

evaluation code

This collects: predictions true labels

```
model.eval()
all_preds = []
all_labels = []
with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        outputs = model(images)
        preds = torch.sigmoid(outputs)
        preds = (preds > 0.5).float()
```

```
all_preds.extend(preds.cpu().numpy())
```

```
all_labels.extend(labels.numpy())
```

```
-----  
-----  
KeyboardInterrupt                                Traceback (most recent call  
last)
```

```
Cell In[66], line 13
```

```
     9     for images, labels in test_loader:  
    10  
    11         images = images.to(device)  
    12  
--> 13         outputs = model(images)  
    14  
    15         preds = torch.sigmoid(outputs)  
    16
```

```
File c:\Users\aaial\AppData\Local\Programs\Python\Python311\Lib\site-  
packages\torch\nn\modules\module.py:1778, in
```

```
Module._wrapped_call_impl(self, *args, **kwargs)
```

```
    1776     return self._compiled_call_impl(*args, **kwargs) # type:  
ignore[misc]
```

```
    1777 else:
```

```
-> 1778     return self._call_impl(*args, **kwargs)
```

```
File c:\Users\aaial\AppData\Local\Programs\Python\Python311\Lib\site-  
packages\torch\nn\modules\module.py:1789, in Module._call_impl(self,  
*args, **kwargs)
```

```
    1784 # If we don't have any hooks, we want to skip the rest of the  
logic in
```

```
    1785 # this function, and just call forward.
```

```
    1786 if not (self._backward_hooks or self._backward_pre_hooks or  
self._forward_hooks or self._forward_pre_hooks
```

```
    1787         or _global_backward_pre_hooks or  
_global_backward_hooks
```

```
    1788         or _global_forward_hooks or  
_global_forward_pre_hooks):
```

```
-> 1789     return forward_call(*args, **kwargs)
```

```
    1791 result = None
```

```
    1792 called_always_called_hooks = set()
```

```
File c:\Users\aaial\AppData\Local\Programs\Python\Python311\Lib\site-  
packages\torchvision\models\resnet.py:285, in ResNet.forward(self, x)
```

```
    284 def forward(self, x: Tensor) -> Tensor:
```

```
--> 285     return self._forward_impl(x)
```

```
File c:\Users\aaial\AppData\Local\Programs\Python\Python311\Lib\site-  
packages\torchvision\models\resnet.py:276, in
```

```

ResNet._forward_impl(self, x)
    274 x = self.layer2(x)
    275 x = self.layer3(x)
--> 276 x = self.layer4(x)
    278 x = self.avgpool(x)
    279 x = torch.flatten(x, 1)

```

```

File c:\Users\aaial\AppData\Local\Programs\Python\Python311\Lib\site-
packages\torch\nn\modules\module.py:1778, in
Module._wrapped_call_impl(self, *args, **kwargs)
    1776     return self._compiled_call_impl(*args, **kwargs) # type:
ignore[misc]
    1777 else:
-> 1778     return self._call_impl(*args, **kwargs)

```

```

File c:\Users\aaial\AppData\Local\Programs\Python\Python311\Lib\site-
packages\torch\nn\modules\module.py:1789, in Module._call_impl(self,
*args, **kwargs)
    1784 # If we don't have any hooks, we want to skip the rest of the
logic in
    1785 # this function, and just call forward.
    1786 if not (self._backward_hooks or self._backward_pre_hooks or
self._forward_hooks or self._forward_pre_hooks
    1787         or _global_backward_pre_hooks or
_global_backward_hooks
    1788         or _global_forward_hooks or
_global_forward_pre_hooks):
-> 1789     return forward_call(*args, **kwargs)
    1791 result = None
    1792 called_always_called_hooks = set()

```

```

File c:\Users\aaial\AppData\Local\Programs\Python\Python311\Lib\site-
packages\torch\nn\modules\container.py:253, in
Sequential.forward(self, input)
    249 """
    250 Runs the forward pass.
    251 """
    252 for module in self:
--> 253     input = module(input)
    254 return input

```

```

File c:\Users\aaial\AppData\Local\Programs\Python\Python311\Lib\site-
packages\torch\nn\modules\module.py:1778, in
Module._wrapped_call_impl(self, *args, **kwargs)
    1776     return self._compiled_call_impl(*args, **kwargs) # type:
ignore[misc]
    1777 else:
-> 1778     return self._call_impl(*args, **kwargs)

```

```

File c:\Users\aaial\AppData\Local\Programs\Python\Python311\Lib\site-

```

```

packages\torch\nn\modules\module.py:1789, in Module._call_impl(self,
*args, **kwargs)
    1784 # If we don't have any hooks, we want to skip the rest of the
    logic in
    1785 # this function, and just call forward.
    1786 if not (self._backward_hooks or self._backward_pre_hooks or
self._forward_hooks or self._forward_pre_hooks
    1787         or _global_backward_pre_hooks or
    _global_backward_hooks
    1788         or _global_forward_hooks or
    _global_forward_pre_hooks):
-> 1789     return forward_call(*args, **kwargs)
    1791 result = None
    1792 called_always_called_hooks = set()

```

File c:\Users\aaial\AppData\Local\Programs\Python\Python311\Lib\site-packages\torchvision\models\resnet.py:92, in BasicBlock.forward(self, x)

```

    89 def forward(self, x: Tensor) -> Tensor:
    90     identity = x
--> 92     out = self.conv1(x)
    93     out = self.bn1(out)
    94     out = self.relu(out)

```

File c:\Users\aaial\AppData\Local\Programs\Python\Python311\Lib\site-packages\torch\nn\modules\module.py:1778, in Module._wrapped_call_impl(self, *args, **kwargs)

```

    1776     return self._compiled_call_impl(*args, **kwargs) # type:
    ignore[misc]
    1777 else:

```

```

-> 1778     return self._call_impl(*args, **kwargs)

```

File c:\Users\aaial\AppData\Local\Programs\Python\Python311\Lib\site-packages\torch\nn\modules\module.py:1789, in Module._call_impl(self, *args, **kwargs)

```

    1784 # If we don't have any hooks, we want to skip the rest of the
    logic in
    1785 # this function, and just call forward.
    1786 if not (self._backward_hooks or self._backward_pre_hooks or
self._forward_hooks or self._forward_pre_hooks
    1787         or _global_backward_pre_hooks or
    _global_backward_hooks
    1788         or _global_forward_hooks or
    _global_forward_pre_hooks):
-> 1789     return forward_call(*args, **kwargs)
    1791 result = None
    1792 called_always_called_hooks = set()

```

File c:\Users\aaial\AppData\Local\Programs\Python\Python311\Lib\site-packages\torch\nn\modules\conv.py:565, in Conv2d.forward(self, input)

```
564 def forward(self, input: Tensor) -> Tensor:
--> 565     return self._conv_forward(input, self.weight, self.bias)
```

File c:\Users\aaial\AppData\Local\Programs\Python\Python311\Lib\site-packages\torch\nn\modules\conv.py:560, in Conv2d._conv_forward(self, input, weight, bias)

```
547 if self.padding_mode != "zeros":
548     return F.conv2d(
549         F.pad(
550             input, self._reversed_padding_repeated_twice,
mode=self.padding_mode
557         self.groups,
558     )
--> 560 return F.conv2d(
561     input, weight, bias, self.stride, self.padding,
self.dilation, self.groups
562 )
```

KeyboardInterrupt:

metrics

```
accuracy = accuracy_score(all_labels, all_preds)
precision = precision_score(all_labels, all_preds)
recall = recall_score(all_labels, all_preds)
f1 = f1_score(all_labels, all_preds)
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1-score:", f1)
```

Save model

```
torch.save(model.state_dict(), 'vehicle_model.pth')
torch.save(model.state_dict(), "traffic_light_model.pth")
torch.save(model.state_dict(), "pedestrian_model.pth")
```